

Migración a AWS de Bajo Costo

<https://gemini.google.com/app/715754f1837ecc25>

User prompt: al repositorio debo subir back y front asi tal cual cierto?

Response: **Sí, subir ambos en un solo repositorio (estilo monorepo) con las carpetas /backend y /frontend es lo más recomendado y estándar para pruebas técnicas**, sobre todo si el formulario pide un solo enlace de GitHub.

Solo ten mucho cuidado con **3 detalles clave** antes de hacer el commit para que no tengas problemas:

1. ¡Ojo con los .git anidados! (El error más común)

Tanto NestJS como Vite suelen ejecutar `git init` automáticamente cuando se crean los proyectos:

- Si la carpeta `backend/` o `frontend/` tienen una carpeta oculta `.git` por dentro y haces commit desde la raíz, GitHub las interpretará como *submódulos* y se verán como carpetas grises con una flecha a las que el evaluador **no podrá entrar a ver el código**.
- **Cómo solucionarlo antes de subir:** Abre la terminal en la raíz (`prueba_desempeño`) y verifica si existen:

```
# Si existen, elimina los .git internos para dejar solo el de la raíz:
Remove-Item -Recurse -Force backend\.git -ErrorAction SilentlyContinue
Remove-Item -Recurse -Force frontend\.git -ErrorAction SilentlyContinue
```

(El único `.git` debe estar en la carpeta raíz del proyecto).

2. Verifica el .gitignore en la raíz

Asegúrate de que el `.gitignore` de la raíz (o dentro de cada carpeta) cubra ambas partes para que no se suban dependencias ni compilados:

```
# Dependencias
node_modules/
backend/node_modules/
frontend/node_modules/
```

```
# Compilados
dist/
build/
backend/dist/
frontend/dist/
```

```
# Secretos
.env
*.env
backend/.env
frontend/.env
!.env.example
```

3. Agrega un README.md en la raíz (Suma muchísimos puntos)

En la raíz crea un `README.md` bien estructurado con los enlaces en vivo en la parte superior:

```
# PlanCity - Despliegue en AWS
```

Aplicación Fullstack desarrollada con NestJS, PostgreSQL y React/Vite, desplegada en arquitectura Serverless y Conte

```
## 🚀 Enlaces en Vivo
- **Frontend (CloudFront / S3):** [URL_DE_TU_CLOUDFRONT]
- **Documentación Swagger / API:** http://13.59.27.78:3000/api/docs
```

```
## 🏗️ Arquitectura AWS
- **Backend:** AWS ECS (Fargate) con contenedor Dockerizado desde Amazon ECR.
- **Base de Datos:** Amazon RDS (PostgreSQL).
- **Frontend:** Amazon S3 + Distribución de Amazon CloudFront.
```

```
## 📁 Estructura del Proyecto
- `/backend`: API construida con NestJS y TypeORM.
- `/frontend`: Interfaz de usuario.
```

Una vez verificado eso, puedes hacer desde la raíz:

```
git init
git add .
git commit -m "feat: initial commit fullstack plancity with aws deployment"
```

¡Y empujarlo a tu repositorio de GitHub!

User prompt: sin querer hice `git init` sin validar lo que dijiste de `git` anidados... y tambien hice `git add .`

Response: No te preocupes, no has hecho el commit, así que tiene solución inmediata.

Sigue estos pasos en tu terminal de PowerShell en la carpeta raíz (prueba_desempeño):

Paso 1: Deshacer el git add . (deseleccionar todo)

Ejecuta este comando para vaciar el área de preparación (staging):

```
git rm -r --cached .
```

(Tus archivos de código están a salvo, esto solo le quita la marca de "listo para commit").

Paso 2: Eliminar las carpetas .git internas (si existen)

Ejecuta estos dos comandos para borrar cualquier historial anidado en el backend y frontend:

```
Remove-Item -Recurse -Force backend\.git -ErrorAction SilentlyContinue
Remove-Item -Recurse -Force frontend\.git -ErrorAction SilentlyContinue
```

Paso 3: Verificar que el .gitignore esté activo en la raíz

Asegúrate de que en la raíz del proyecto (prueba_desempeño) exista tu archivo .gitignore con al menos estas líneas:

```
node_modules/
backend/node_modules/
frontend/node_modules/
dist/
backend/dist/
frontend/dist/
.env
backend/.env
frontend/.env
!.env.example
```

Paso 4: Volver a agregar y verificar

1. Agrega de nuevo los archivos limpios:

```
git add .
```

2. Revisa el estado de Git:

```
git status
```

¿Cómo saber si quedó bien? En la lista verde que te arroja git status, debes ver archivos individuales como:

- new file: backend/src/main.ts
- new file: frontend/src/App.tsx

Si vieras algo como new file: backend (modified content, untracked content), aún habría un .git dentro. Si te lista los archivos normalmente, ya puedes proceder con el commit:

```
git commit -m "feat: initial commit plancity fullstack"
```

User prompt: PS C:\Users\Adria\Desktop\prueba_desempeño> git rm -r --cached . error: the following file has staged content different from both the file and the HEAD: backend (use -f to force removal) PS C:\Users\Adria\Desktop\prueba_desempeño> git rm -r --cached fatal: No pathspec was given. Which files should I remove?

Response: Ese mensaje apareció justamente porque backend tenía su propio .git interno y Git lo detectó como un submódulo. Solo debes agregar la bandera -f (force) para obligar a Git a sacarlo del área de preparación sin tocar tus archivos de código:

```
git rm -r -f --cached .
```

Una vez ejecutado, limpia las carpetas de Git internas y vuelve a agregar el proyecto:

1. Eliminar los .git anidados

```
Remove-Item -Recurse -Force .\backend\.git -ErrorAction SilentlyContinue
Remove-Item -Recurse -Force .\frontend\.git -ErrorAction SilentlyContinue
```

2. Agregar los archivos limpios

```
git add .
```

3. Verificar el estado

```
git status
```

En la salida de `git status` debes ver la lista de archivos individuales (como `backend/src/...` y `frontend/src/...`) en verde. Si todo sale así, ya puedes hacer tu commit con tranquilidad:

```
git commit -m "feat: initial commit plancity fullstack"
```

User prompt: pero ven aca cada carpeta quedo con un gitignore... asi debe quedar?

Response: Sí, está perfectamente bien y es totalmente válido en Git.

Git funciona de forma jerárquica:

- El `.gitignore` dentro de `backend/` protege automáticamente todo lo que esté dentro de la carpeta `backend/` (ignora sus propios `node_modules/`, `dist/`, `.env`).
- El `.gitignore` de `frontend/` hace lo mismo para la interfaz.

Puedes dejarlos así sin ningún problema. Solo asegúrate de que también exista un `.gitignore` en la raíz (`prueba_desempeño/.gitignore`) como red de seguridad general.

Para comprobar que tus `.gitignore` están haciendo su trabajo, corre:

```
git status
```

Si en la lista verde **no** ves carpetas `node_modules` ni archivos `.env` (solo ves tus archivos fuente como `src/`, `package.json`, `Dockerfile`, etc.), todo está funcionando como debe y ya puedes proceder con:

```
git commit -m "feat: initial commit plancity fullstack"
```

User prompt: dame un prompt para agy para que me haga un readme del proyecto sin que me toque el codigo para que no dañe nada jajaja

Response: Copia y pega este prompt directamente en el chat de tu asistente:

Actúa como un Tech Lead y documentador técnico senior. Tu única tarea es inspeccionar la estructura de este proyecto

REGLA CRÍTICA DE SEGURIDAD:

- TIENES PROHIBIDO modificar, refactorizar, formatear, mover o eliminar cualquier archivo de código existente (.ts, .js, etc.).
- Únicamente debes crear/actualizar el archivo `README.md` en la raíz. Nada más.

Contexto del Proyecto:

- Nombre: PlanCity (Prueba de desempeño Fullstack).
- Arquitectura: Monorepo con carpetas `/backend` y `/frontend`.
- Backend: NestJS, TypeORM, Node.js, validaciones, autenticación con JWT.
- Base de Datos: PostgreSQL alojada en Amazon RDS.
- Infraestructura Backend: Dockerfile multi-stage, imagen en Amazon ECR, servicio ejecutado en Amazon ECS con AWS Fargate.
- Frontend: Aplicación web moderna (React / Vite) preparada para despliegue en Amazon S3 y distribución global con AWS CloudFront.

Estructura que debe tener el `README.md`:

1. Título del Proyecto con insignias/badges representativos (NestJS, React, AWS, Docker, PostgreSQL, TypeScript).
2. Descripción ejecutiva de la solución y objetivos de la prueba técnica.
3. Enlaces en vivo:
 - Frontend (CloudFront / S3): [Dejar placeholder evidente para pegar la URL]
 - Backend API & Documentación Swagger: `http://13.59.27.78:3000/api/docs`
4. Diagrama textual o lista de Arquitectura Cloud en AWS (ECS Fargate, ECR, RDS PostgreSQL, S3, CloudFront).
5. Estructura de directorios del monorepo (`/backend`, `/frontend`) explicando brevemente la responsabilidad de cada uno.
6. Guía de instalación y ejecución en local paso a paso:
 - Prerrequisitos (Node.js, Docker, PostgreSQL).
 - Configuración de variables de entorno (referenciando los archivos `.env.example`).
 - Comandos para instalar dependencias, correr migraciones TypeORM y ejecutar en modo desarrollo tanto para backend como frontend.
7. Endpoints clave y módulos implementados (Auth, usuarios, planes/ciudades, etc., según lo que encuentres en el código).
8. Decisiones técnicas y consideraciones de seguridad (SSL en RDS con `NODE_TLS_REJECT_UNAUTHORIZED``, roles de IAM, etc.).

Genera el contenido en formato Markdown estándar de GitHub sin alterar ningún otro archivo del workspace.

User prompt: ya le di git add . y no me toma el readme

Response: No lo tomó porque tu terminal está parada dentro de la carpeta `backend` (`C:\Users\Adria\Desktop\prueba_desempeño\backend`).

Cuando ejecutas `git add .`, Git solo agrega los archivos que están en el directorio donde estás ubicado y sus subcarpetas hacia abajo; como el `README.md` está en la raíz (`../README.md`), se queda por fuera.

Ejecuta estos comandos en tu terminal para solucionarlo:

```
cd ..
git add .
git status
```

Al hacer `cd ..` subirás a la raíz del proyecto (`prueba_desempeño`) y `git add .` preparará el `README.md` de inmediato. En `git status` ya te debe aparecer en verde como `new file: README.md`.

User prompt: y por que el frontend tampoco está en verde?

Response: En la terminal **sí está en verde y ya está tomado por Git**: fíjate que toda la lista que tienes abierta abajo dice `new file: ../frontend/...`

La confusión con el explorador visual de VS Code a la izquierda ocurre por dos razones:

1. El significado del punto verde en VS Code

En VS Code, el punto verde o la letra U (como la que tiene el `README.md`) no significa "agregado a Git", sino **Untracked (Sin rastrear)**:

- `README.md` y `backend` tienen el indicador verde porque tienen archivos sueltos que Git aún **no** tiene en seguimiento.
- `frontend` **ya NO tiene el punto verde** precisamente porque **ya le diste `git add`**: sus archivos pasaron al área de preparación (*Staged / Index*). Al estar ya rastreados, VS Code les retira la advertencia de "archivos nuevos sin rastrear".

2. Desfase de refresco en la interfaz de VS Code

Cuando ejecutas comandos de Git manualmente desde la terminal integrada, la interfaz visual del explorador de carpetas a veces tarda unos segundos en sincronizar el estado gráfico.

Para forzar a VS Code a sincronizarse con lo que hiciste en la terminal:

1. Haz clic en el ícono de **Control de código fuente** en la barra lateral izquierda (el tercer ícono de arriba hacia abajo, el que tiene una bifurcación de ramas y un número 1).
2. En la parte superior de ese panel, haz clic en el botón de **Refrescar** (la flecha circular). Verás cómo todos los archivos de `frontend` aparecen listados bajo la sección **Cambios en el área de preparación (Staged Changes)**.

Para dejarlo 100% listo con el README

Sube a la raíz y agrega el `README.md` que quedó pendiente:

```
cd ..
git add .
git status
```

En la salida de `git status` verás absolutamente todo (`backend`, `frontend` y `README.md`) listado en verde como `new file:`, confirmando que el paquete completo está listo para tu commit.

User prompt: como es que me conecto al repo local?

Response: Si te refieres a **vincular tu repositorio local con un repositorio nuevo en GitHub** para subir el código:

1. Crea el repositorio vacío en GitHub:

- Entra a github.com/new.
- Asigne un nombre (por ejemplo, `plancity-fullstack`).
- **Importante:** Déjalo completamente vacío (no marques las casillas de *Add a README file*, *.gitignore* ni *license*).
- Haz clic en **Create repository** y copia la URL HTTPS (terminada en `.git`).

2. Haz el primer commit en tu máquina (desde la raíz `prueba_desempeño`):

```
git commit -m "feat: initial commit plancity fullstack"
```

3. Renombra la rama a `main`:

```
git branch -M main
```

4. Conecta tu repositorio local con la URL de GitHub:

```
git remote add origin https://github.com/TU_USUARIO/TU_REPOSITORIO.git
```

5. Sube el código a GitHub:

```
git push -u origin main
```

Si únicamente te referías a **navegar por la terminal a la carpeta raíz de tu repositorio local**:

```
cd C:\Users\Adria\Desktop\prueba_desempeño
```

Y para verificar que Git reconoce la carpeta activa y su historial:

```
git status
```

User prompt: donde uno ve que usuarios podria ingresar de prueba para loguearme en la app?

Response: Tienes **3 formas principales** de encontrarlos o tener credenciales listas de inmediato:

1. En los archivos de Seeds del Backend (*Donde ves el correo y la contraseña en texto plano*)

En la base de datos las contraseñas se guardan encriptadas (\$2b\$. . .), por lo que el mejor lugar para ver la contraseña real es el código donde se insertaron:

1. En VS Code, presiona `Ctrl + P`.
2. Escribe `seed` o busca en la carpeta:
 - `backend/src/database/seeds/`
 - `backend/src/seeds/` (o cualquier archivo llamado `seed.ts` o `user.seed.ts`).
3. Abre ese archivo: allí verás los objetos creados por defecto con su correo y contraseña original (ej. `admin@plancity.com`, `Password123!`).

2. Registrar un usuario nuevo en Swagger (*La más rápida y 100% segura*)

Si no encuentras las contraseñas en los seeds o la base de datos está vacía, crea uno en 10 segundos:

1. Entra a tu Swagger: `http://13.59.27.78:3000/api/docs`
2. Despliega el módulo de **Auth** y busca el endpoint `POST /api/auth/register` (o `/auth/register / /users`).
3. Haz clic en **Try it out**, ingresa un JSON con datos de prueba (correo y contraseña que conozcas) y dale a **Execute**.
4. Una vez responda `201 Created`, ya tienes un usuario real guardado en RDS listo para iniciar sesión en tu frontend.

3. Consultar los correos existentes en la Base de Datos

Si ejecutaste semillas o migraciones y quieres ver qué correos ya existen:

1. En tu gestor (DBeaver o pgAdmin), conéctate a la base de datos `plancity`.
2. Abre un editor SQL y consulta:

```
SELECT id, email, role FROM "user";
```

(o `SELECT id, email, role FROM users;` según cómo se llame tu tabla).
3. Te listará todos los correos registrados. Si son usuarios predefinidos de prueba, habitualmente usan contraseñas estándar de desarrollo como `Admin1234*`, `Password123!`, `123456` o `Admin123`.

4. En el documento o PDF de la prueba

Revisa el PDF o documento con los requerimientos de la prueba técnica; muchas veces en la sección de autenticación los evaluadores indican explícitamente: *"Utilizar las credenciales por defecto: admin@test.com / Admin2024"*.

User prompt: una pregunta... me di cuenta que el fronten salia asi en naranja y no en verde justo en el archivo que se editó anteriormente... será que si guradó el cambio? porque sigue asi y ya hace rato le hice push al repo

Response: **Sí, el cambio está 100% guardado y ya está en tu repositorio de GitHub.**

En tu propia pantalla se puede confirmar por dos detalles directos:

1. **La línea de GitLens (Prueba irrefutable):** En la línea 27, el texto gris claro a la derecha dice textualmente:

```
Usted, hace 20 minutos • plancity fullstack ....
```

Esa etiqueta solo aparece cuando esa línea exacta ya forma parte del commit que realizaste (`plancity fullstack`).
2. **Archivo guardado:** En la pestaña superior `eventDetail.tsx` aparece una **X** y no un círculo blanco (●), lo que confirma que no hay modificaciones pendientes de guardar en disco.

¿Por qué aparece en naranja y con los puntos en las carpetas?

Ese color naranja **no proviene de Git**, sino del analizador de TypeScript / ESLint de VS Code:

- Al lado derecho del nombre `eventDetail.tsx` aparece el número **6**.
- En la barra inferior tienes abierta la pestaña **Problemas 6**.
- VS Code pinta de color naranja/amarillo las carpetas (`pages`, `events`, `features`, `src`, `frontend`) para avisarte de que dentro de esa ruta hay un archivo con **6 advertencias (warnings) de tipado o sintaxis** (como el uso de `f: any`, variables no utilizadas o tipos implícitos).

Si quieres comprobarlo por ti mismo en la nube, entra a tu repositorio en **GitHub desde el navegador**, abre la ruta `frontend/src/features/events/pages/eventDetail.tsx` y verás esa misma función ya subida.
